



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Machine Learning

Práctica 7

Perceptrón Multicapa

Profesor: Joel Omar Juárez Gambino

Integrantes de equipo:

Alvarez Peñaloza Nathan Arturo

Juárez Utrilla Edwin Josué

López Lin Billy Yong Le

Miranda Santiago Gustavo Ángel

Grupo: 6 CM 1

Introducción

El reconocimiento y clasificación de números escritos a mano es un problema interesante en el campo de la inteligencia artificial y la ciencia de datos. Los dígitos manuscritos pueden utilizarse en una amplia variedad de aplicaciones, como la lectura de cheques, la entrada de datos en formularios y escritura mediante un lápiz óptico a texto plano. Por lo tanto, a lo largo de esta práctica, se entrenará un modelo de Machine Learning que permita identificar a qué clase (números del 0 al 9) pertenece cada imagen de entrada; para ello, se utilizará el perceptrón multicapa.

El reconocimiento óptico de caracteres es una innovadora tecnología que sustenta decenas de herramientas que uso diario; explicado de manera sencilla, se trata de un tipo de software que traduce documentos digitalizados a un formato legible por computadora.

Sin el reconocimiento y clasificación de números escritos a mano, el ordenador interpretaría cada documento digitalizado como una sola imagen, del mismo modo que vemos las fotografías. En ese formato, la computadora no es capaz de reconocer las letras, palabras o frases individuales, lo que limita las maneras en que esta y, en última instancia, los usuarios, pueden interactuar con el documento.

El software de digitalización, reconocimiento y clasificación de números escritos a mano permite que el computador vea un documento digitalizado del mismo modo que ve los documentos de texto. De este modo, la máquina puede interactuar con los archivos digitalizados de la misma forma que con los documentos digitales originales. Esto incluye:

- Usar las funciones de búsqueda
- Editar
- Usar herramientas de análisis y comparación
- Procesar, almacenar, recuperar y compartir información

El reconocimiento óptico de caracteres se puede utilizar para traducir texto impreso e incluye dos procesos relacionados, siendo estos diseñados para capturar datos manuscritos y marcados por humanos:

- Reconocimiento inteligente de caracteres (ICR): es el proceso de capturar y traducir caracteres escritos y stampados a mano, como los que aparecen en formularios estructurados.
- Reconocimiento óptico de marcas (OMR): es el proceso de capturar datos marcados por humanos como líneas o áreas sombreadas en formularios, como encuestas, cuestionarios y exámenes con preguntas de opción múltiple.

El perceptrón multicapa es un modelo de aprendizaje supervisado, por lo que se entrena mediante un conjunto de datos etiquetados que se utilizan para enseñar al modelo a realizar una tarea específica. El modelo se ajusta a través del proceso de entrenamiento, ajustando los pesos de cada una de sus conexiones para minimizar el error en sus predicciones. Es una variante del perceptrón básico, un tipo de modelo de redes neuronales empleado en la clasificación y predicción de clases.

En el perceptrón multicapa, se utilizan varias capas de neuronas que trabajan juntas para completar una tarea específica. Estas neuronas son unidades de procesamiento que reciben entradas, efectúan cálculos y producen una salida. La capa de entrada recibe los datos de entrada y se los proporciona a las capas siguientes, que realizan cálculos y producen salidas. La capa final, o capa de salida, produce la salida final del modelo. Se caracteriza por tener salidas disjuntas pero relacionadas entre sí, de tal manera que la salida de una neurona es la entrada de la siguiente.

En el perceptrón multicapa se pueden diferenciar 2 fases:

1. Propagación en la que se calcula el resultado de salida de la red desde los valores de entrada hacia delante.
2. Aprendizaje en la que los errores obtenidos a la salida del perceptrón se van propagando hacia atrás (backpropagation) con el objetivo de modificar los pesos de las conexiones para que el valor estimado de la red se asemeje cada vez más al real, esta aproximación se realiza mediante la función gradiente del error.

Resultados del programa

Se modificaron varios de los parámetros del clasificador del perceptrón multicapa con la ayuda de GrindSearch para disminuir el número de pruebas que se hubieran necesitado de haberlo hecho una por una, de estas pruebas se seleccionó la mejor combinación de parámetros para dar el mejor resultado, tanto los mejores parámetros como los resultados se muestran a continuación.

Training Set

Parámetros seleccionados

Parámetro		Valor
alpha		0.2
hidden_layer_sizes	Layer 1	300 neuronas
	Layer 2	150 neuronas
	Layer 3	75 neuronas
activation		relu
max_iter		100
solver		adam

Reporte de clasificación

Accuracy de entrenamiento: 0.9961166666666667					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	5923	
1	1.00	1.00	1.00	6742	
2	1.00	0.99	1.00	5958	
3	0.99	1.00	0.99	6131	
4	1.00	1.00	1.00	5842	
5	1.00	0.99	1.00	5421	
6	1.00	1.00	1.00	5918	
7	1.00	1.00	1.00	6265	
8	1.00	1.00	1.00	5851	
9	1.00	0.99	0.99	5949	
accuracy			1.00	60000	
macro avg	1.00	1.00	1.00	60000	
weighted avg	1.00	1.00	1.00	60000	

Test Set

Reporte de clasificación

```
-----Uso del conjunto de pruebas-----
Cargando conjunto de prueba...

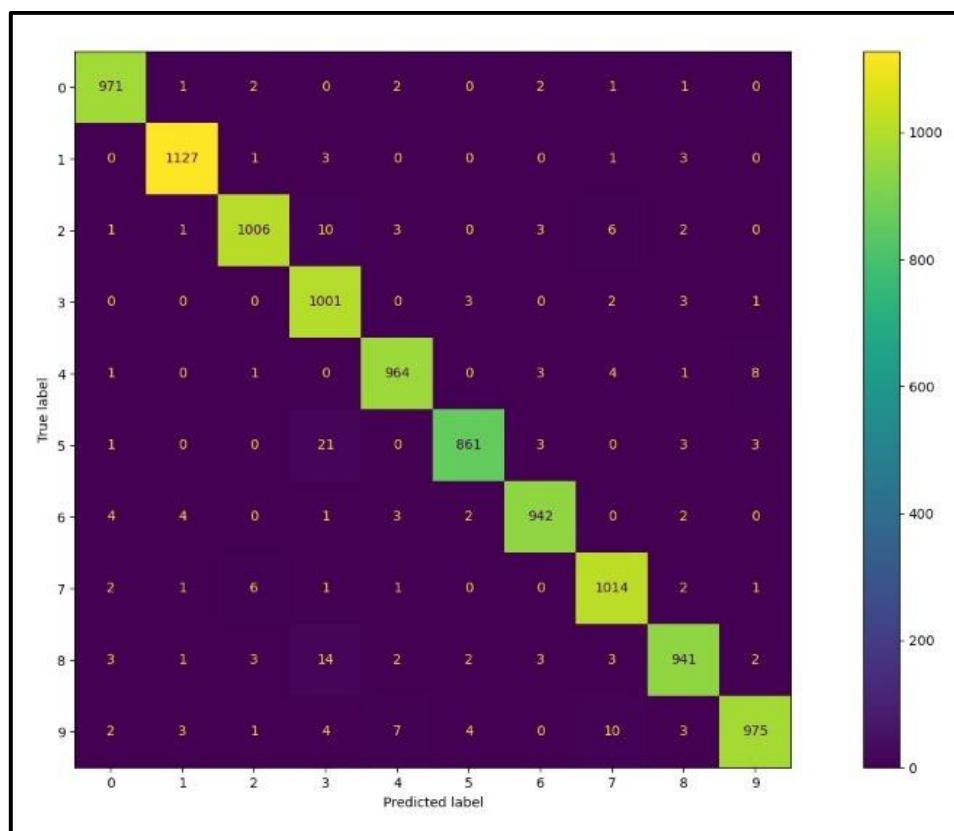
Accuracy de las pruebas: 0.9802
      precision    recall  f1-score   support

     0       0.99       0.99       0.99        980
     1       0.99       0.99       0.99       1135
     2       0.99       0.97       0.98       1032
     3       0.95       0.99       0.97       1010
     4       0.98       0.98       0.98        982
     5       0.99       0.97       0.98        892
     6       0.99       0.98       0.98        958
     7       0.97       0.99       0.98       1028
     8       0.98       0.97       0.97        974
     9       0.98       0.97       0.98       1009

   accuracy          0.98       10000
  macro avg          0.98       10000
 weighted avg          0.98       10000

Cantidad de errores: 198
```

Matriz de confusión

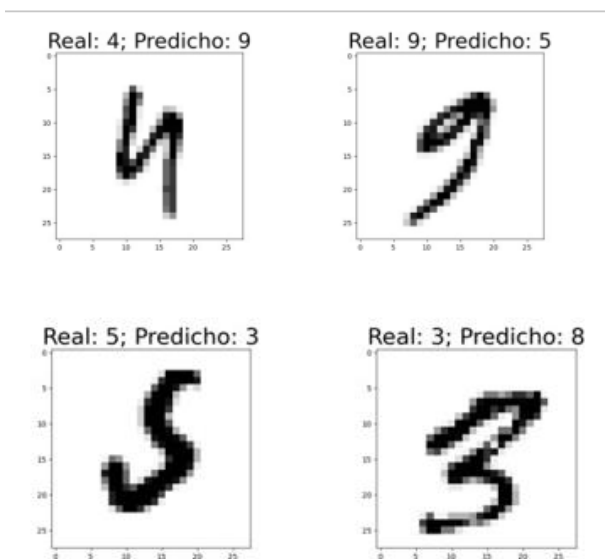


Conclusiones

Tras haber completado el modelo de clasificación de dígitos, se observó un comportamiento más que adecuado a lo largo de todo el proceso de ajuste de los parámetros, pues todas las métricas de rendimiento (en especial la precisión) mantuvieron valores bastante altos en todo momento, incluso durante las primeras pruebas del modelo, partiendo de los parámetros por default en la mayoría de ellos. Esto nos lleva a concluir que este modelo de aprendizaje es ideal para este tipo de problemas de clasificación. Por tanto, ajustando mejor algunos parámetros como el número de capas ocultas y neuronas, el alpha, activation, solver y número de iteraciones, los resultados presentaron un progreso notable, pasando de un 90% de exactitud a un 98%, permitiéndonos apreciar lo útil que resulta ajustar los parámetros correctamente.

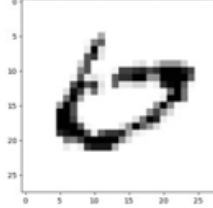
No obstante, es importante mencionar aquel porcentaje de resultados fallidos, los cuales tras analizar las imágenes de los dígitos que el modelo no clasificó correctamente, se describen a continuación.

Un caso es el número “9”, ya que la curvatura superior confunde al modelo cuando esta presenta una apertura muy marcada, o cuando el lado que sobresale en el lado inferior presenta una curvatura considerable. Otra situación similar se presenta con el número “8”, cuya curvatura y apertura lo hace fácilmente confundible con el número 3 por ejemplo.

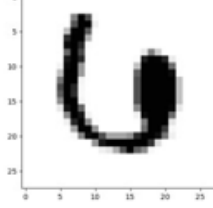


Para el número “6”, en la mayoría de los casos donde lo clasifica mal, tiende a interpretarlo como un 8, debido a la cantidad de curvas en ambos números; por otro lado, la sección superior que sobresale se puede interpretar como parte de la curva del 0.

Real: 6; Predicho: 0

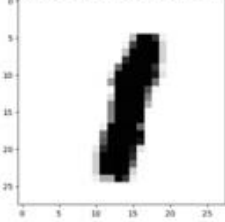


Real: 6; Predicho: 0

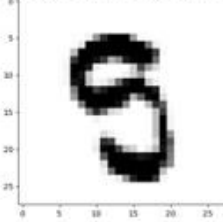


El número “5” es la clase en la que más errores se cometió, y esto se observa fácilmente en el reporte de clasificación, puesto que cumple con las características descritas en los dos casos anteriores: una gran cantidad de curvas y segmentos casi cerrados. Esto provoca que este número sea confundido con varios números, como el 9, 3, 6, u 8.

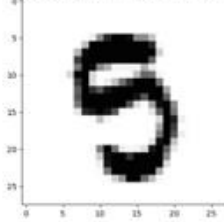
Real: 1; Predicho: 8



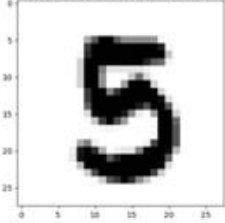
Real: 5; Predicho: 9



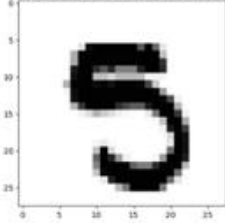
Real: 5; Predicho: 9



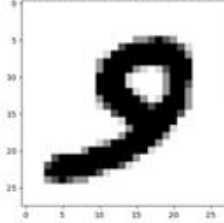
Real: 5; Predicho: 3



Real: 5; Predicho: 3

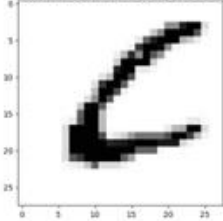


Real: 9; Predicho: 3

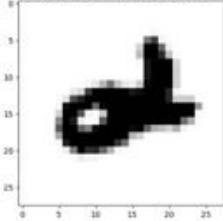


Otro aspecto relevante de señalar es que gran parte de los errores identificados se originan debido a que el modelo encuentra números amorfos o que, de manera intencional, presentan una cierta similitud con otros números o incluso símbolos, esto con el propósito de confundir al sistema.

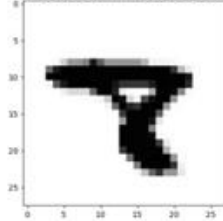
Real: 6; Predicho: 0

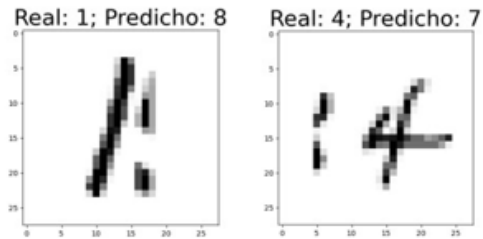


Real: 2; Predicho: 4



Real: 8; Predicho: 7





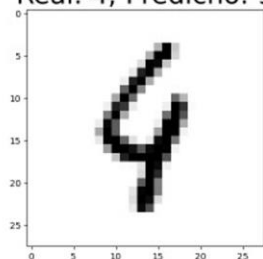
Teniendo en cuenta lo anterior, se puede considerar como una posible mejora hacer el uso de un preprocesamiento de datos para mejorar la calidad de las imágenes o aumentar de pixeles por imagen.

Adicionalmente, debido a que en algunos casos el error se presenta porque el número a identificar muestra cierta inclinación, eso ya produciendo errores o confusiones en la clasificación, interesaría realizar una implementación de volver a entrenar el modelo con los mismos datos, pero en distintas posiciones y rotaciones, también permitiría un incremento en el rendimiento y la disminución de errores

De manera general, se puede concluir que este tipo de clasificador es bastante bueno para el problema de reconocimiento de dígitos escritos a mano, e incluso para clasificar otro tipo de imágenes ajenos a manuscritos, puesto que el alto rendimiento que demostró en el entrenamiento y en las pruebas son sorprendentes, considerando el nivel de complejidad de la clasificación.

Predicciones fallidas

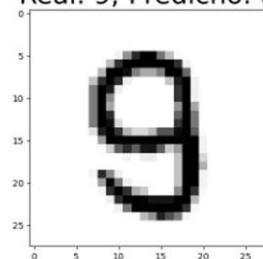
Real: 4; Predicho: 9



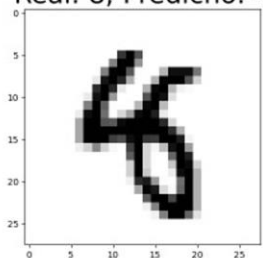
Real: 2; Predicho: 7



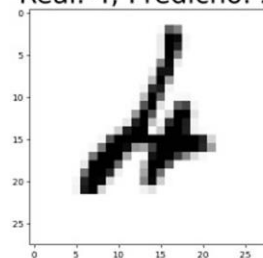
Real: 9; Predicho: 8



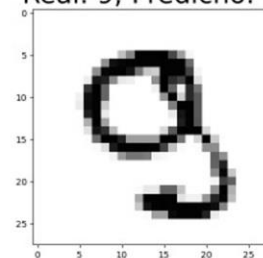
Real: 8; Predicho: 4



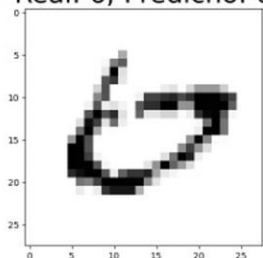
Real: 4; Predicho: 2



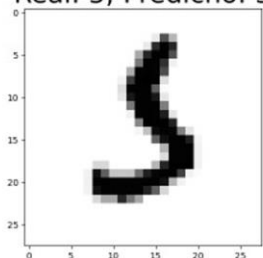
Real: 9; Predicho: 8



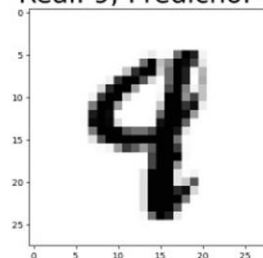
Real: 6; Predicho: 0



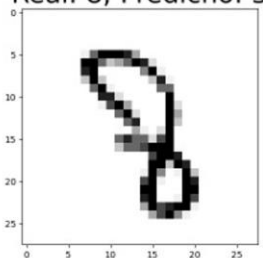
Real: 5; Predicho: 3



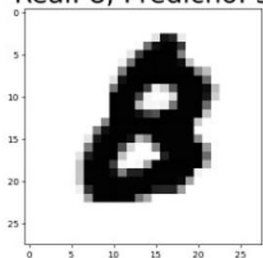
Real: 9; Predicho: 4



Real: 8; Predicho: 3



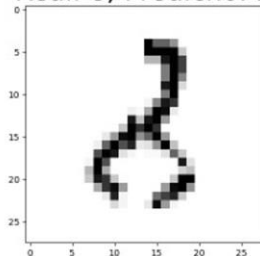
Real: 8; Predicho: 3



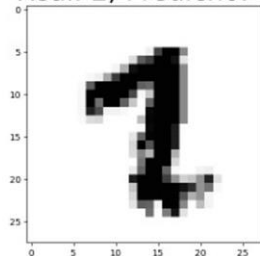
Real: 3; Predicho: 5



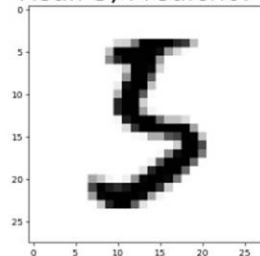
Real: 8; Predicho: 2



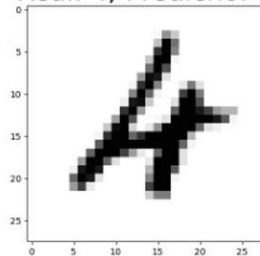
Real: 2; Predicho: 7



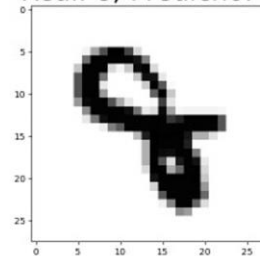
Real: 5; Predicho: 3



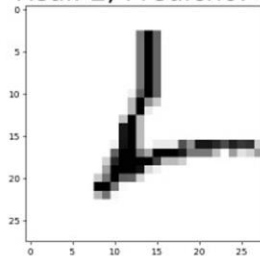
Real: 4; Predicho: 6



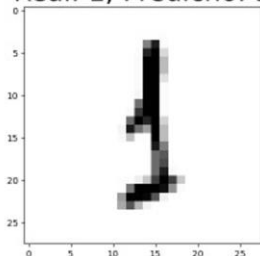
Real: 8; Predicho: 4



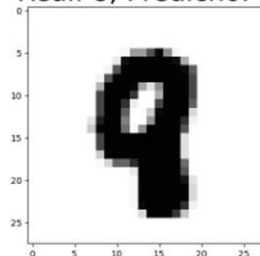
Real: 2; Predicho: 6



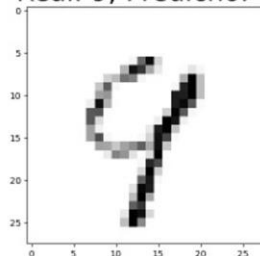
Real: 1; Predicho: 3



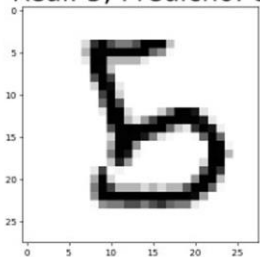
Real: 8; Predicho: 9



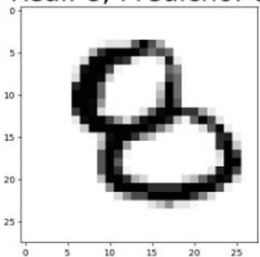
Real: 9; Predicho: 7



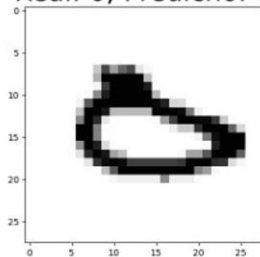
Real: 5; Predicho: 8



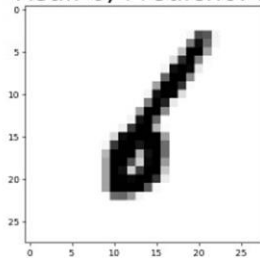
Real: 8; Predicho: 6



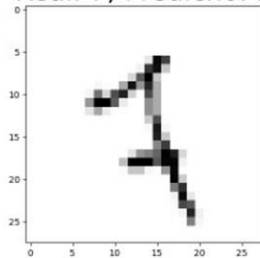
Real: 0; Predicho: 6



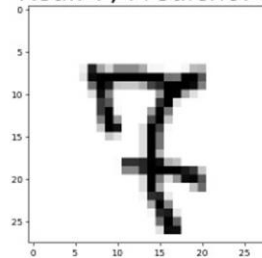
Real: 6; Predicho: 1



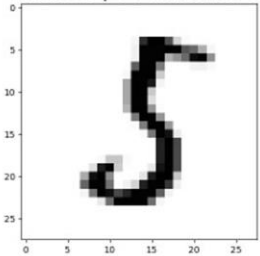
Real: 7; Predicho: 2



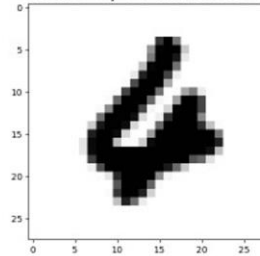
Real: 7; Predicho: 8



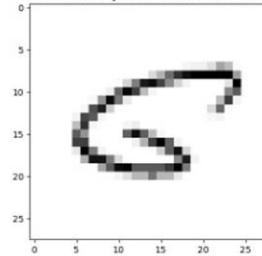
Real: 5; Predicho: 3



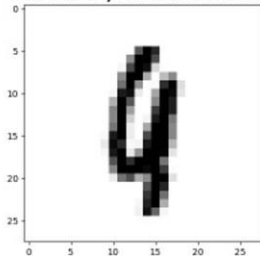
Real: 4; Predicho: 6



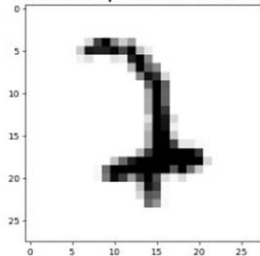
Real: 6; Predicho: 5



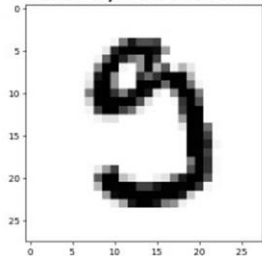
Real: 9; Predicho: 1



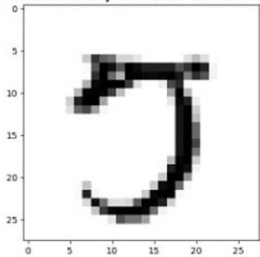
Real: 7; Predicho: 2



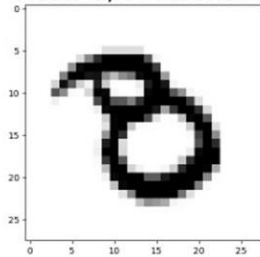
Real: 9; Predicho: 5



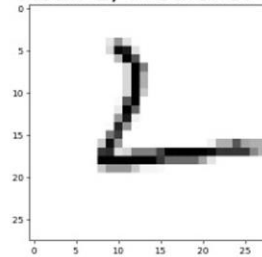
Real: 5; Predicho: 3



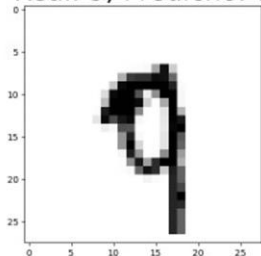
Real: 8; Predicho: 3



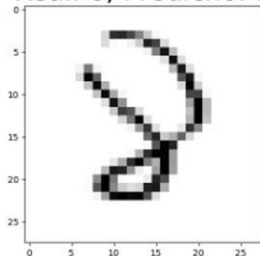
Real: 2; Predicho: 6



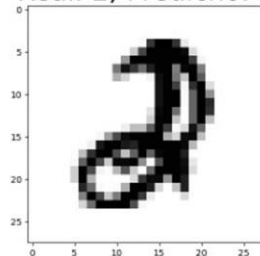
Real: 9; Predicho: 7



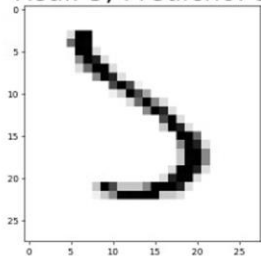
Real: 8; Predicho: 3



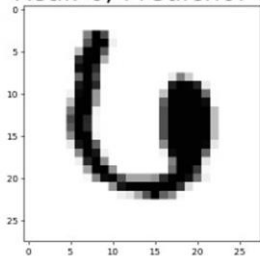
Real: 2; Predicho: 3



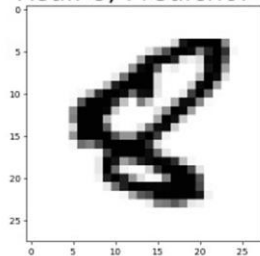
Real: 5; Predicho: 3



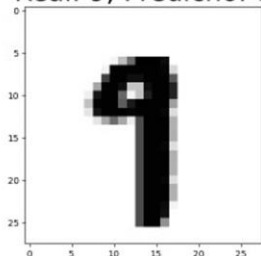
Real: 6; Predicho: 0



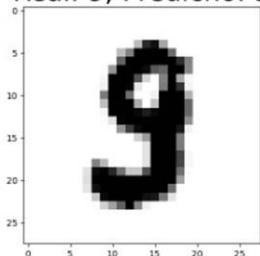
Real: 8; Predicho: 6



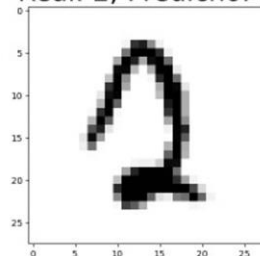
Real: 9; Predicho: 8



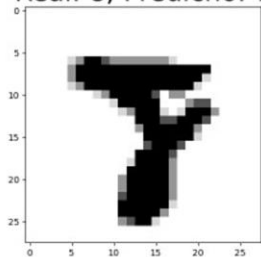
Real: 9; Predicho: 3



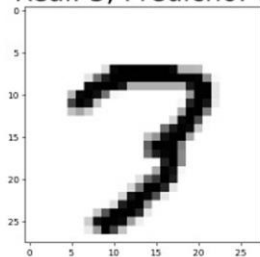
Real: 2; Predicho: 1



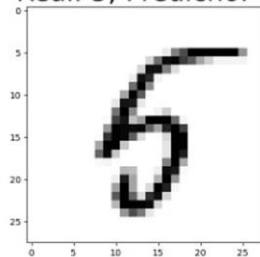
Real: 8; Predicho: 7



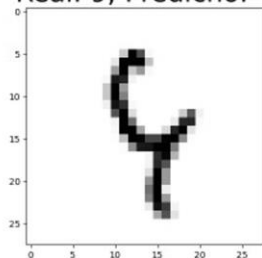
Real: 3; Predicho: 7



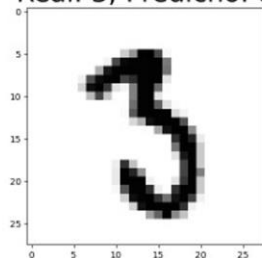
Real: 5; Predicho: 6



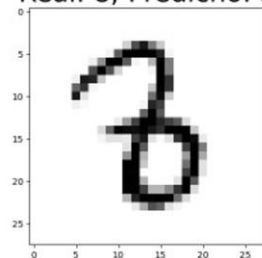
Real: 9; Predicho: 4



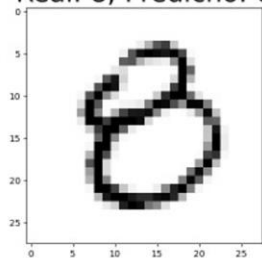
Real: 3; Predicho: 8



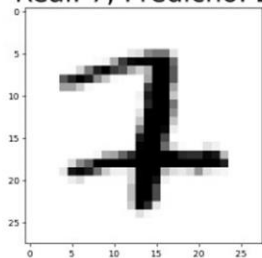
Real: 8; Predicho: 3



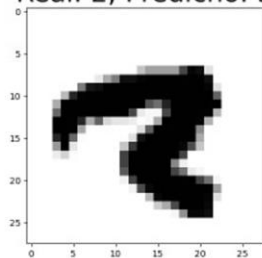
Real: 8; Predicho: 0



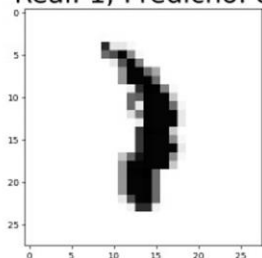
Real: 7; Predicho: 2



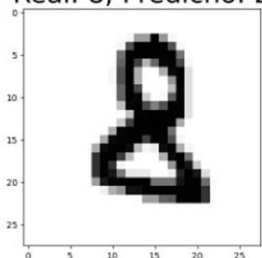
Real: 2; Predicho: 3



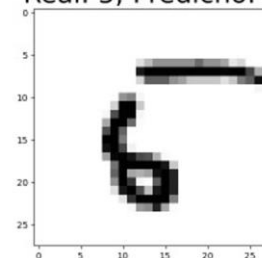
Real: 1; Predicho: 8



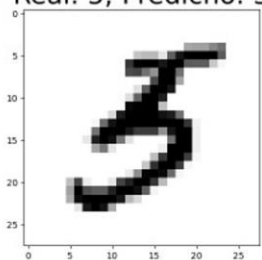
Real: 8; Predicho: 2



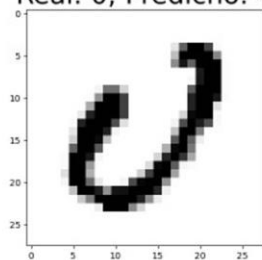
Real: 5; Predicho: 6



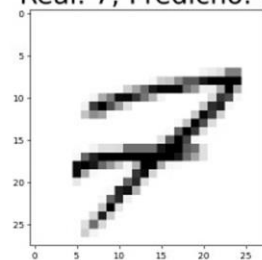
Real: 5; Predicho: 3



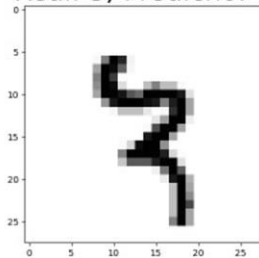
Real: 0; Predicho: 4



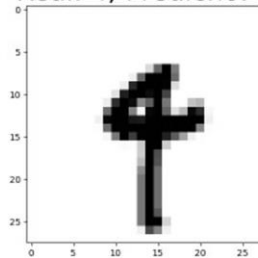
Real: 7; Predicho: 9



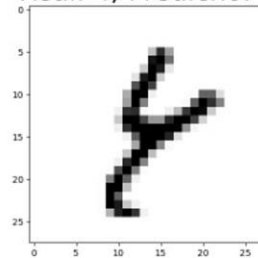
Real: 3; Predicho: 7



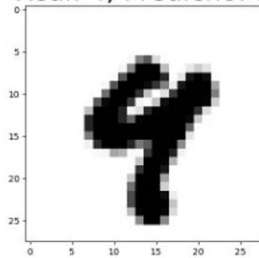
Real: 4; Predicho: 9



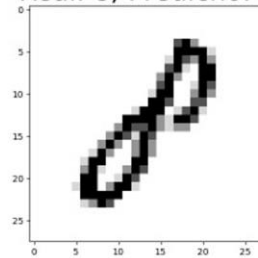
Real: 4; Predicho: 8



Real: 4; Predicho: 9



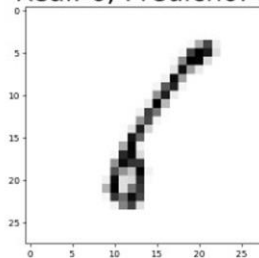
Real: 8; Predicho: 1



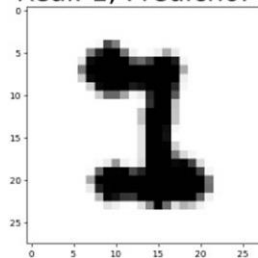
Real: 2; Predicho: 0



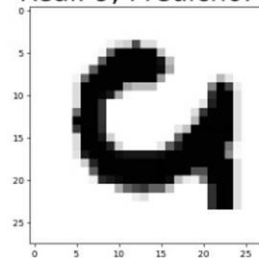
Real: 6; Predicho: 1



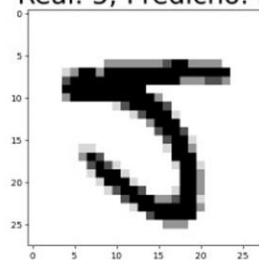
Real: 1; Predicho: 3



Real: 9; Predicho: 0



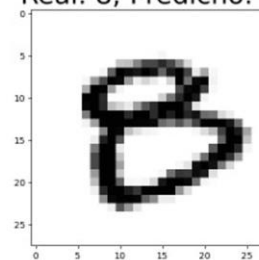
Real: 5; Predicho: 3



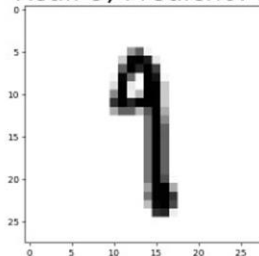
Real: 0; Predicho: 2



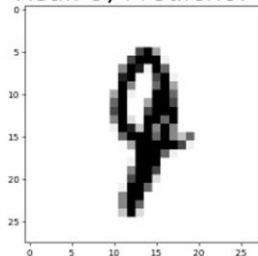
Real: 8; Predicho: 0



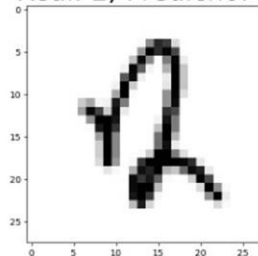
Real: 9; Predicho: 1



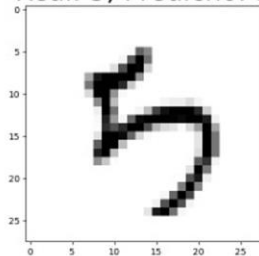
Real: 9; Predicho: 7



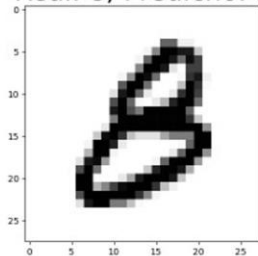
Real: 2; Predicho: 4



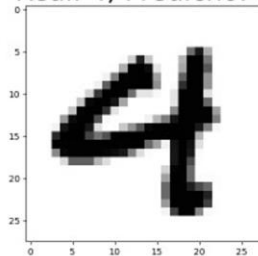
Real: 5; Predicho: 3



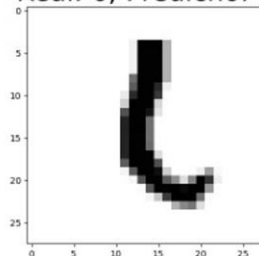
Real: 8; Predicho: 3



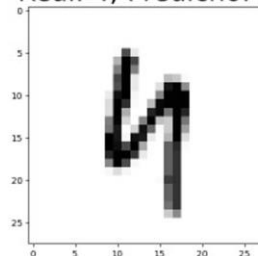
Real: 4; Predicho: 9



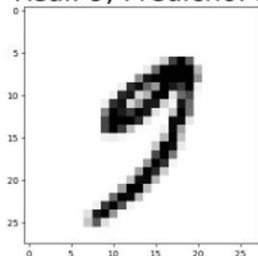
Real: 6; Predicho: 1



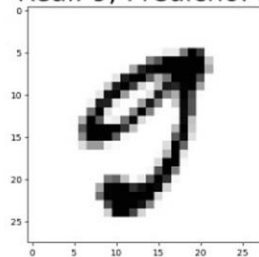
Real: 4; Predicho: 9



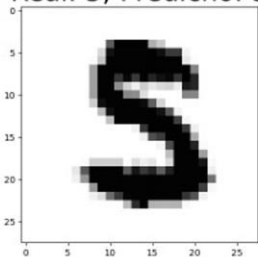
Real: 9; Predicho: 5



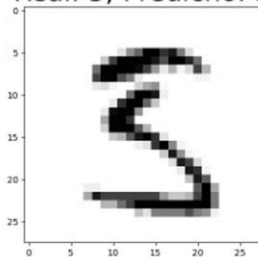
Real: 9; Predicho: 5



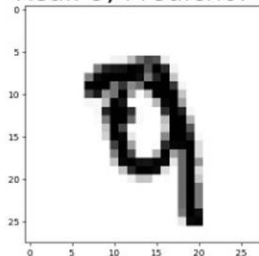
Real: 5; Predicho: 3



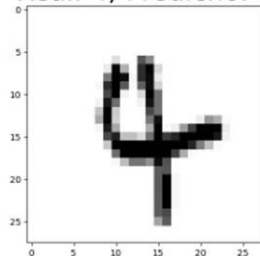
Real: 5; Predicho: 3



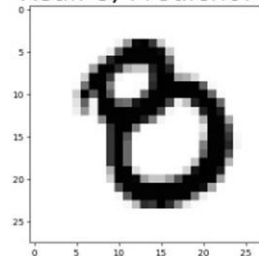
Real: 9; Predicho: 7



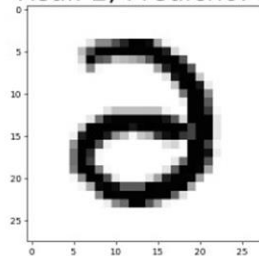
Real: 4; Predicho: 7



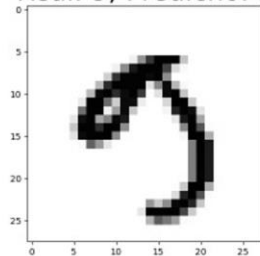
Real: 8; Predicho: 0



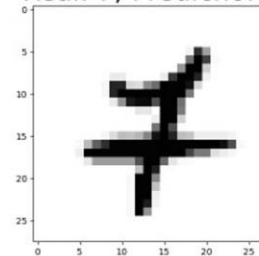
Real: 2; Predicho: 3



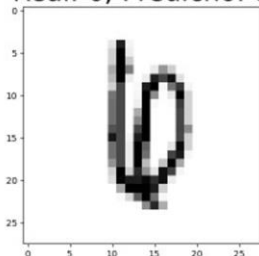
Real: 9; Predicho: 3



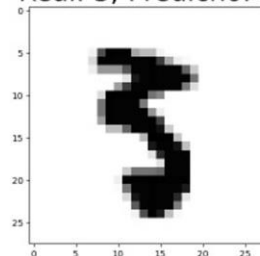
Real: 7; Predicho: 4



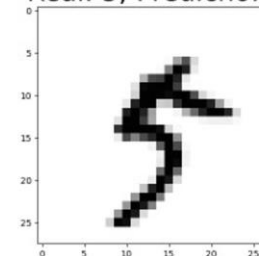
Real: 6; Predicho: 8



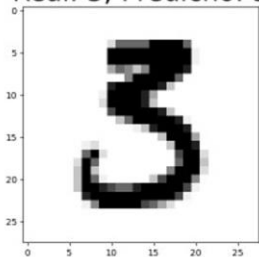
Real: 3; Predicho: 5



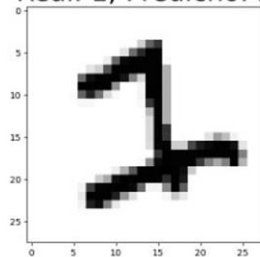
Real: 5; Predicho: 9



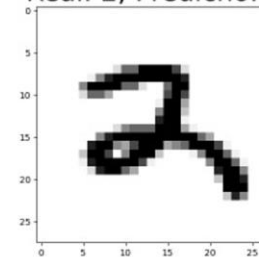
Real: 3; Predicho: 5



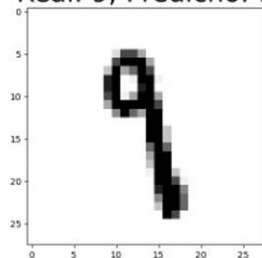
Real: 1; Predicho: 2



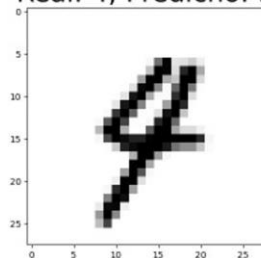
Real: 2; Predicho: 3



Real: 9; Predicho: 1



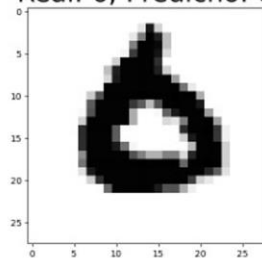
Real: 4; Predicho: 9



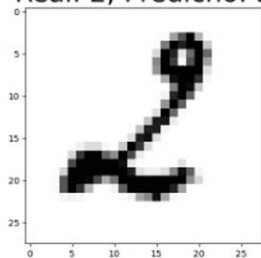
Real: 8; Predicho: 3



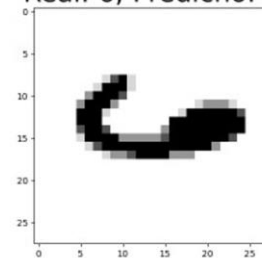
Real: 6; Predicho: 0



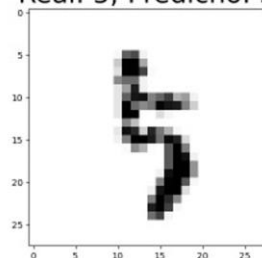
Real: 2; Predicho: 3



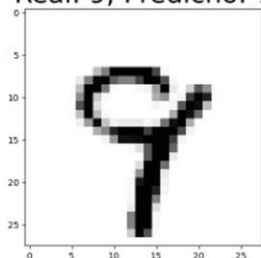
Real: 6; Predicho: 4



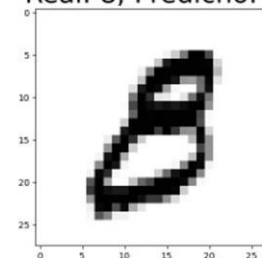
Real: 5; Predicho: 3



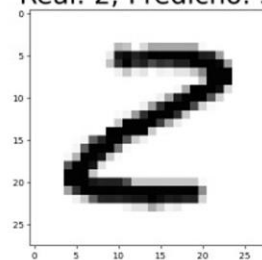
Real: 9; Predicho: 7



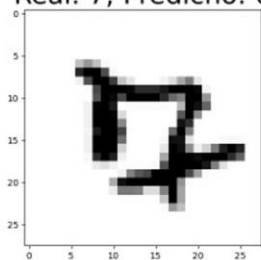
Real: 8; Predicho: 3



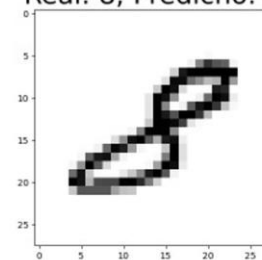
Real: 2; Predicho: 3



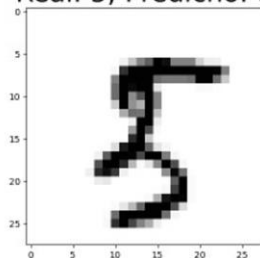
Real: 7; Predicho: 0



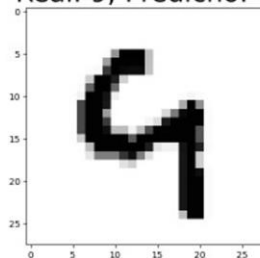
Real: 8; Predicho: 3



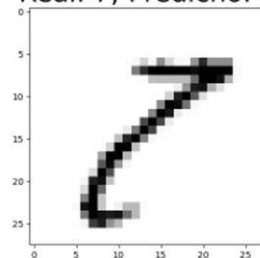
Real: 5; Predicho: 8



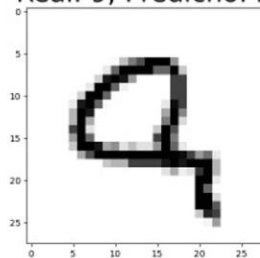
Real: 9; Predicho: 4



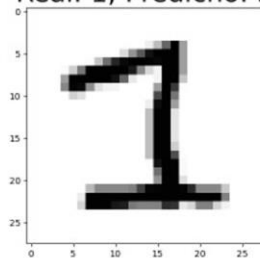
Real: 7; Predicho: 8



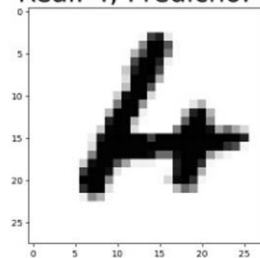
Real: 9; Predicho: 2



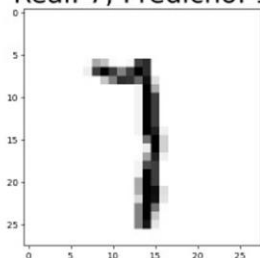
Real: 1; Predicho: 3



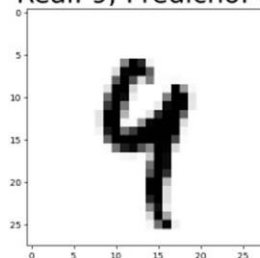
Real: 4; Predicho: 6



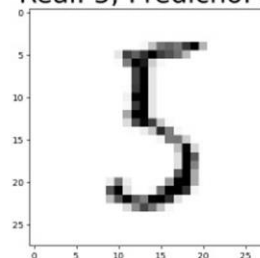
Real: 7; Predicho: 1



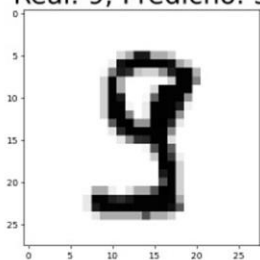
Real: 9; Predicho: 4



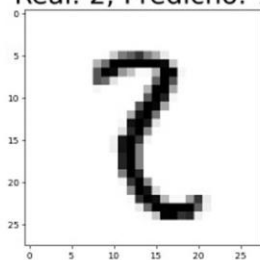
Real: 5; Predicho: 3



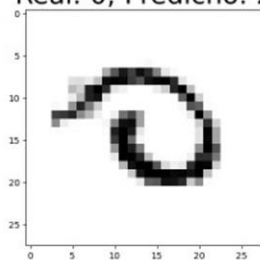
Real: 9; Predicho: 3



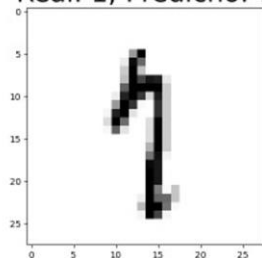
Real: 2; Predicho: 7



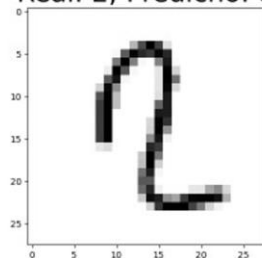
Real: 0; Predicho: 2



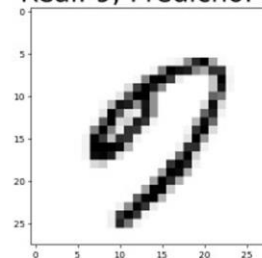
Real: 1; Predicho: 7



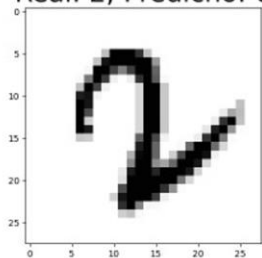
Real: 2; Predicho: 8



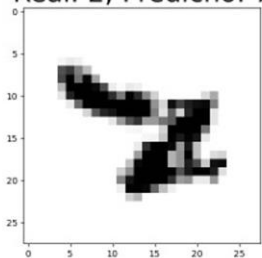
Real: 9; Predicho: 0



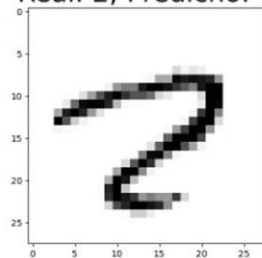
Real: 2; Predicho: 6



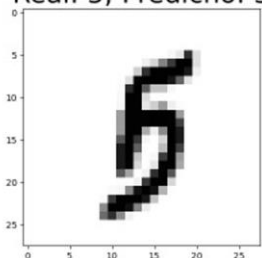
Real: 2; Predicho: 7



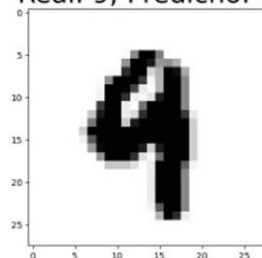
Real: 2; Predicho: 7



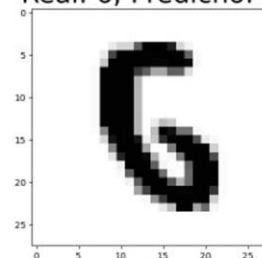
Real: 5; Predicho: 3



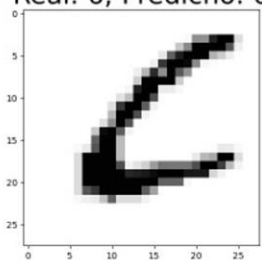
Real: 9; Predicho: 4



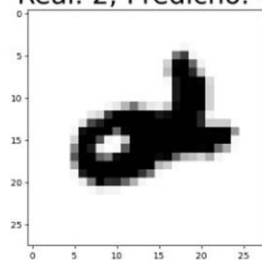
Real: 6; Predicho: 5



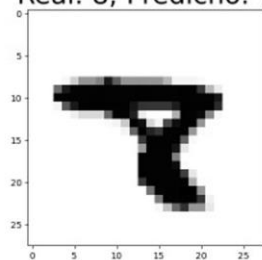
Real: 6; Predicho: 0



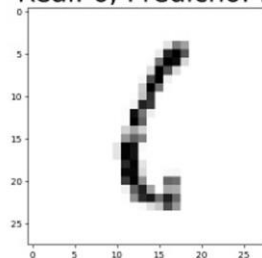
Real: 2; Predicho: 4



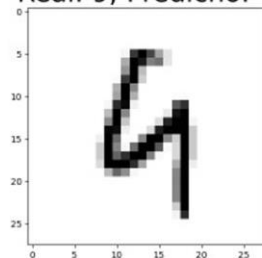
Real: 8; Predicho: 7



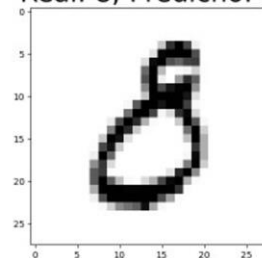
Real: 6; Predicho: 1



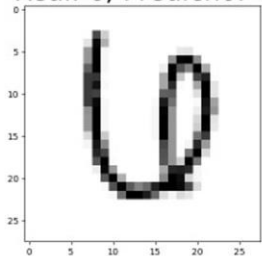
Real: 9; Predicho: 4



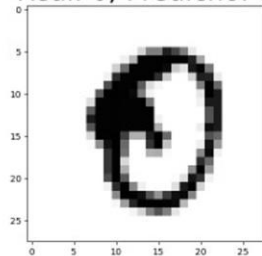
Real: 8; Predicho: 3



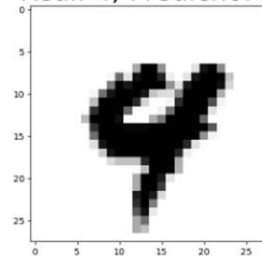
Real: 6; Predicho: 4



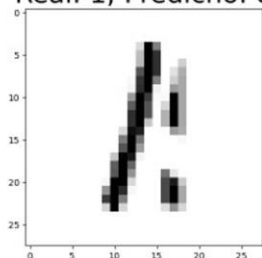
Real: 0; Predicho: 8



Real: 4; Predicho: 9



Real: 1; Predicho: 8



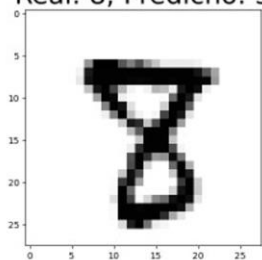
Real: 9; Predicho: 7



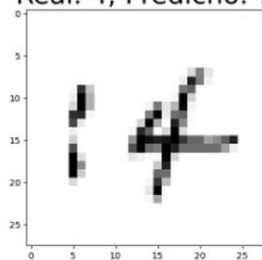
Real: 8; Predicho: 3



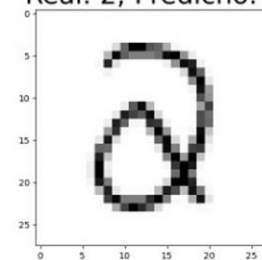
Real: 8; Predicho: 3



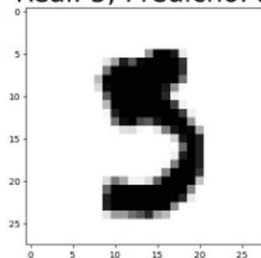
Real: 4; Predicho: 7



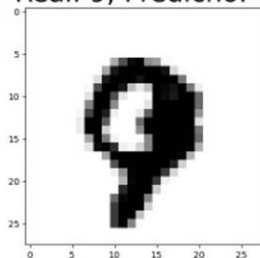
Real: 2; Predicho: 3



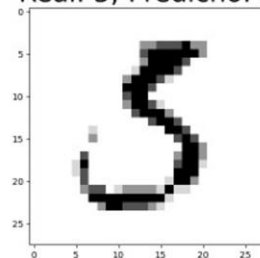
Real: 5; Predicho: 3



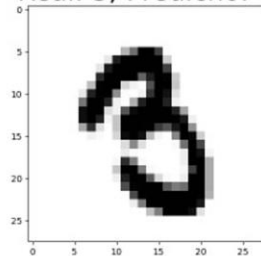
Real: 9; Predicho: 7



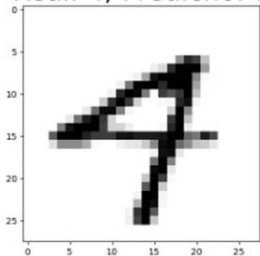
Real: 5; Predicho: 3



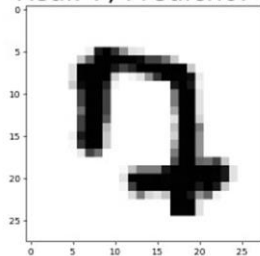
Real: 3; Predicho: 8



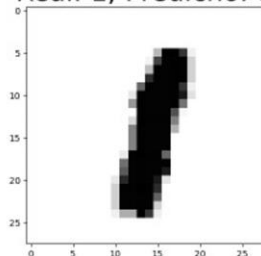
Real: 4; Predicho: 7



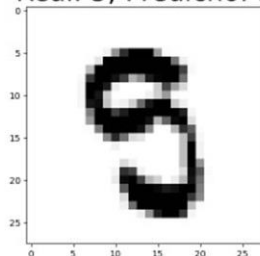
Real: 7; Predicho: 0



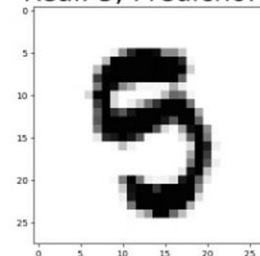
Real: 1; Predicho: 8



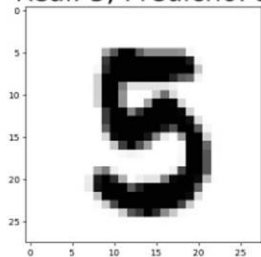
Real: 5; Predicho: 9



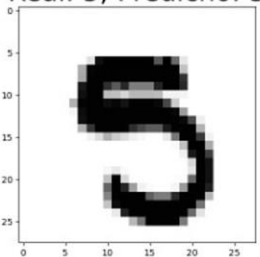
Real: 5; Predicho: 9



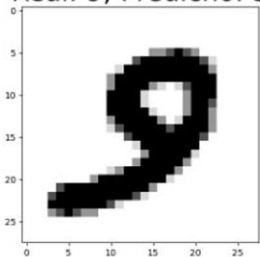
Real: 5; Predicho: 3



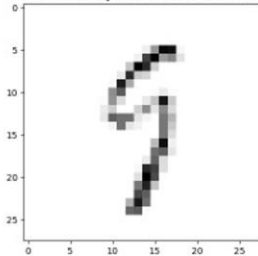
Real: 5; Predicho: 3



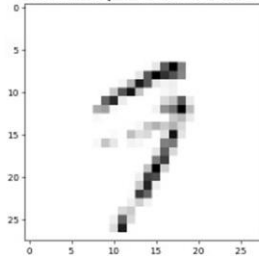
Real: 9; Predicho: 3



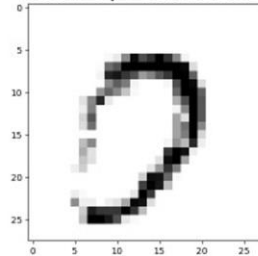
Real: 9; Predicho: 5



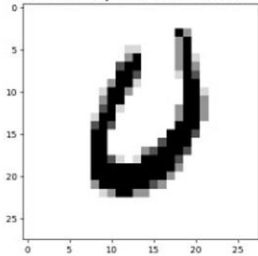
Real: 9; Predicho: 7



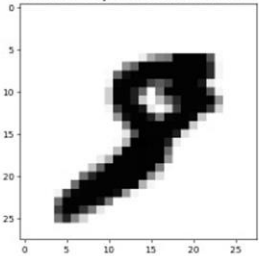
Real: 0; Predicho: 7



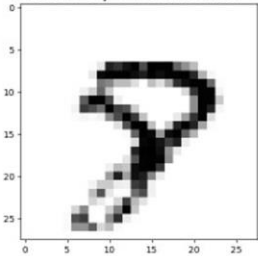
Real: 0; Predicho: 6



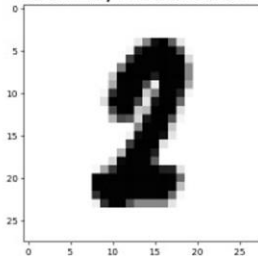
Real: 8; Predicho: 7



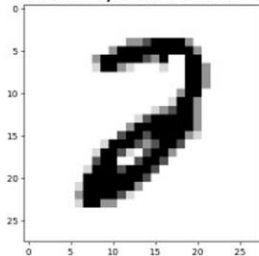
Real: 8; Predicho: 9



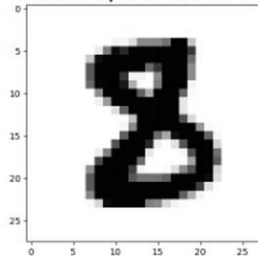
Real: 2; Predicho: 8



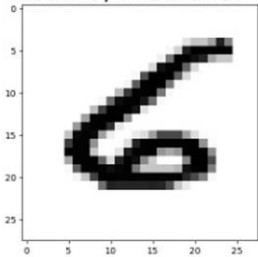
Real: 2; Predicho: 3



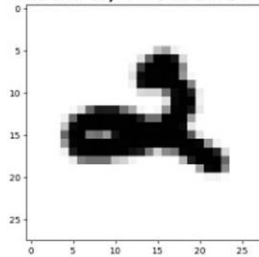
Real: 8; Predicho: 3



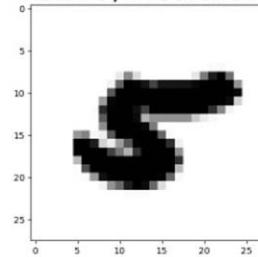
Real: 6; Predicho: 4



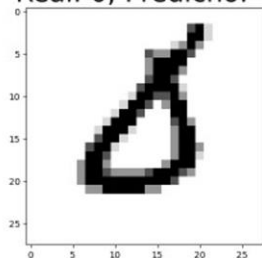
Real: 2; Predicho: 4



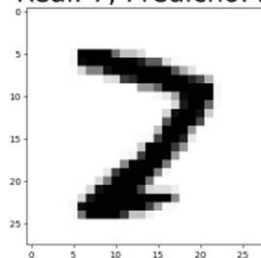
Real: 5; Predicho: 8



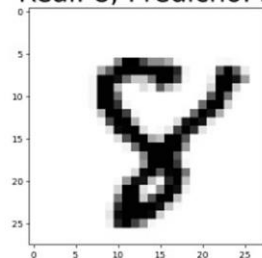
Real: 0; Predicho: 4



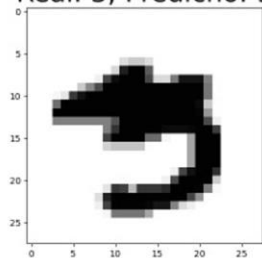
Real: 7; Predicho: 2



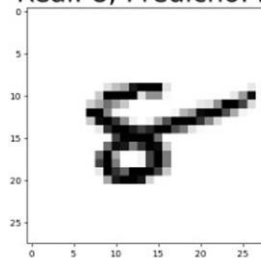
Real: 8; Predicho: 5



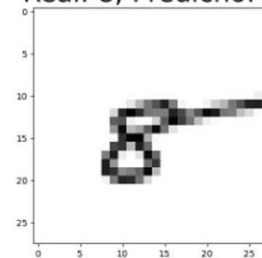
Real: 3; Predicho: 9



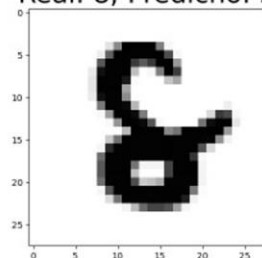
Real: 8; Predicho: 2



Real: 8; Predicho: 6



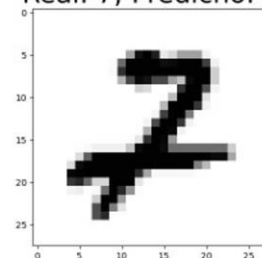
Real: 8; Predicho: 5



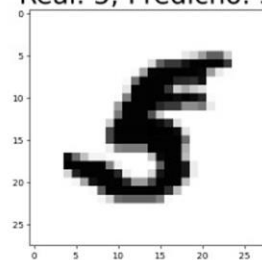
Real: 5; Predicho: 3



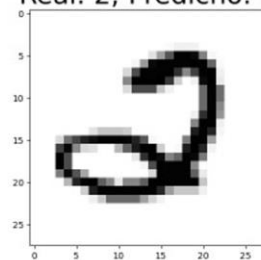
Real: 7; Predicho: 2



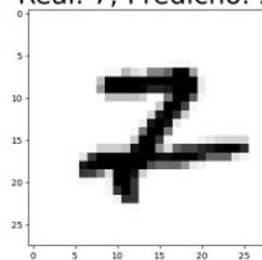
Real: 5; Predicho: 3



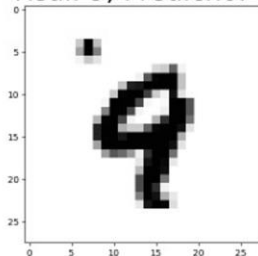
Real: 2; Predicho: 3



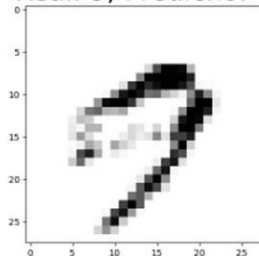
Real: 7; Predicho: 2



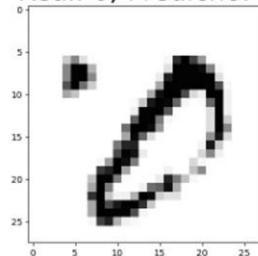
Real: 9; Predicho: 4



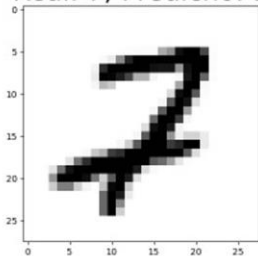
Real: 9; Predicho: 7



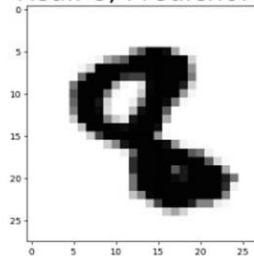
Real: 0; Predicho: 1



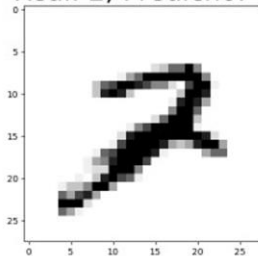
Real: 7; Predicho: 3



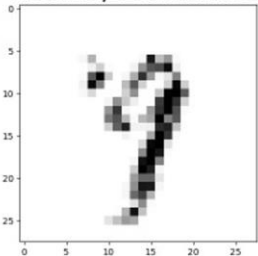
Real: 8; Predicho: 3



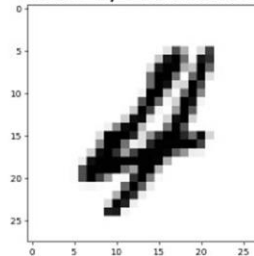
Real: 2; Predicho: 7



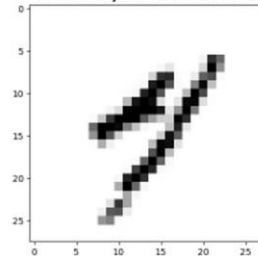
Real: 9; Predicho: 7



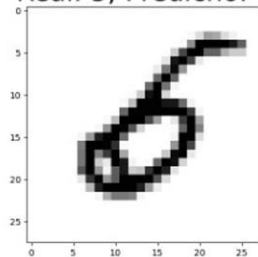
Real: 4; Predicho: 0



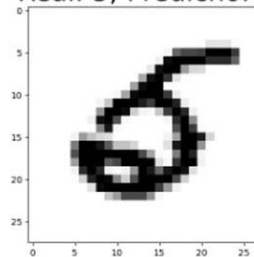
Real: 4; Predicho: 7



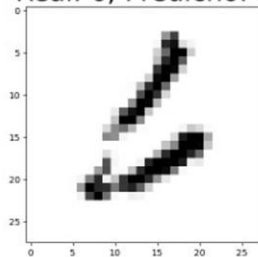
Real: 5; Predicho: 6



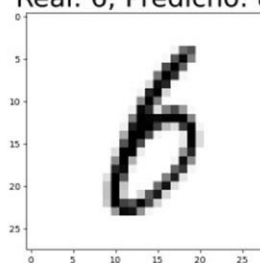
Real: 5; Predicho: 3



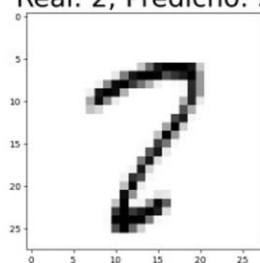
Real: 6; Predicho: 3



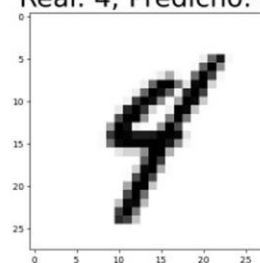
Real: 6; Predicho: 8



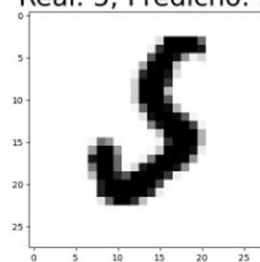
Real: 2; Predicho: 3



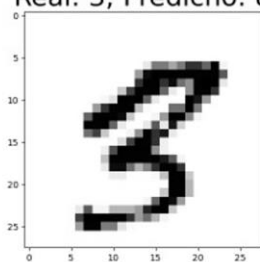
Real: 4; Predicho: 9



Real: 5; Predicho: 3



Real: 3; Predicho: 8



Real: 5; Predicho: 0

